SOLID MAPPING
==========

1. SRP (Single Responsibility Principle)
   Where applied : PaymentService, OrderService, NotificationService
   How   : Payment handles money only; Order owns lifecycle only;
                   Notification reacts to events. IdempotencyGuard, StockGate,
                   ReservationManager, ExpiryWorker each have one job.

2. OCP (Open/Closed Principle)
   Where applied : PricingStrategy, PaymentProvider, DeliveryPartner
   How   : Add FlashSalePricing, UpiAdapter or a new courier by adding
                   a class + config; checkout and order code unchanged.

3. LSP (Liskov Substitution Principle)
   Where applied : RazorpayAdapter, StripeAdapter, ResilientProvider
   How   : All honour the PaymentProvider contract (idempotent charge,
                   status, refund; same error types), so any can replace
                   another, including the decorator.

4. ISP (Interface Segregation Principle)
   Where applied : PaymentProvider (charge/status/refund), StockGate,
                   EventPublisher, OrderRepository
   How   : Small role interfaces; notification code never sees payment
                   write APIs; read-only ReservationQuery is separate from
                   ReservationCommand.

5. DIP (Dependency Inversion Principle)
   Where applied : Services depend on InventoryRepository, PaymentProvider,
                   EventPublisher, StockGate
   How   : Concrete Postgres/Redis/Kafka/gateway classes are injected
                   at the composition root; tests use in-memory fakes.